Universidad Nacional Autónoma De México
Facultad De Ciencias, 2027-I
Fundamentos De Bases De Datos



Apoyo Práctica05:

Restaurar backup de una BD en Docker.

Profesor:
Gerardo Avilés Rosas

Ayudantes de teoría:
Oscar Daniel Flores Linares
Claudia Itzel Gutiérrez Sánchez

Ayudantes de laboratorio:
Ricardo Badillo Macías
Rocío Aylin Huerta González

Objetivo

En este documento veremos como restaurar un backup de una de base de datos en Docker. Para apoyar al alumno para que pueda revisar los materiales que suben en la clase.

Pasos para restaurar a traves de sql: (Ejemplo Linux)

i. Nos vamos a la terminal de nuestro sistema operativo y checamos nuestros contenedores con:

```
sudo docker ps -a
```

```
ricardo@Richardo:~$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED        STATUS
               PORTS                                       NAMES
54eeed7dd8d8   postgres      "docker-entrypoint.s…"   2 months ago   Up 21 hours
               0.0.0.0:5432->5432/tcp, :::5432->5432/tcp   postgres
db95079adf40   hello-world   "/hello"                 2 months ago   Exited (0)
2 months ago                                               admiring_morse
```

ii. Obtenemos el **CONTAINER ID** de la imagen de postgres, en este caso es *54eeed7dd8d8*

iii. Una vez que sepamos el **CONTAINER ID** de la imagen de postgres, debemos checar nuestras bases de datos primero, conectandonos a psql de postgres.

```
sudo docker exec -it 54eeed7dd8d8 psql -U postgres
```

iv. Una vez adentro de psql, escribimos lo siguiente:

```
\l
```

```
ricardo@Richardo:~$ sudo docker exec -it 54eeed7dd8d8 psql -U postgres
psql (15.2 (Debian 15.2-1.pgdg110+1))
Type "help" for help.

postgres=# \l
                                                List of databases
   Name    |  Owner   | Encoding |  Collate   |   Ctype    | ICU Locale | Locale Provider |   Access privileges
-----------+----------+----------+------------+------------+------------+-----------------+-----------------------
 postgres  | postgres | UTF8     | en_US.utf8 | en_US.utf8 |            | libc            |
 template0 | postgres | UTF8     | en_US.utf8 | en_US.utf8 |            | libc            | =c/postgres
   +
           |          |          |            |            |            |                 | postgres=CTc/postgres
 template1 | postgres | UTF8     | en_US.utf8 | en_US.utf8 |            | libc            | =c/postgres
   +
           |          |          |            |            |            |                 | postgres=CTc/postgres
(3 rows)

postgres=#
```

v. Con esto sabremos todas nuestras bases de datos, template0 y template1, son información que necesita postgres, diccionarios y eso. Una vez que checamos el nombre de nuestra base de datos salimos con:

```
\q
```

vi. Para realizar el backup es necesario tener la base de datos vacia, por si se llega a tener tablas repetidas, para asegurarnos de eso crearemos una nueva base de datos el cual llamaremos ejemplo:

```
sudo docker exec -it 54eeed7dd8d8 createdb -U postgres ejemplo
```

vii. Una vez creado nos conectamos a esa nueva base:

```
sudo docker exec -it 54eeed7dd8d8 psql -U postgres ejemplo
```

viii. Checamos el contenido de la base con:

```
\dt
```

Deberia salirnos lo siguiente:

```
ricardo@Richardo:~$ sudo docker exec -it 54eeed7dd8d8 dropdb -U postgres ejemplo
ricardo@Richardo:~$ sudo docker exec -it 54eeed7dd8d8 createdb -U postgres ejemplo
ricardo@Richardo:~$ sudo docker exec -it 54eeed7dd8d8 psql -U postgres ejemplo
psql (15.2 (Debian 15.2-1.pgdg110+1))
Type "help" for help.

ejemplo=# \dt
Did not find any relations.
ejemplo=#
```

ix. Para restaurar el backup utilizaremos $<$ el cual tenemos que especificar la dirección donde se encuentra el backup.

```
sudo docker exec -i 54eeed7dd8d8 psql -U postgres -d ejemplo <
    /home/ricardo/Descargas/backup/ejemplo.sql
```

```
ricardo@Richardo:~$ sudo docker exec -i 54eeed7dd8d8 psql -U postgres -d ejemplo < /home/ricardo/Descargas/bac
kup/ejemplo.sql
SET
SET
SET
SET
SET
 set_config
------------

(1 row)

SET
SET
SET
SET
ALTER SCHEMA
COMMENT
SET
SET
CREATE TABLE
ALTER TABLE
```

x. Si todo salio bien, al conectarnos a la base de datos ejemplo con psql y checar las tablas de nuestra base de datos, deberia aparecenos algo parecido:

```
ricardo@Richardo:~$ sudo docker exec -it 54eeed7dd8d8 psql -U postgres ejemplo
psql (15.2 (Debian 15.2-1.pgdg110+1))
Type "help" for help.

ejemplo=# \dt
              List of relations
 Schema |       Name        | Type  |  Owner
--------+-------------------+-------+----------
 public | componerse        | table | postgres
 public | contar            | table | postgres
 public | deshabilitar      | table | postgres
 public | dias              | table | postgres
 public | especialidad      | table | postgres
 public | estacion          | table | postgres
 public | examenmedico      | table | postgres
 public | incidente         | table | postgres
 public | licencia          | table | postgres
 public | linea             | table | postgres
 public | lineaincidente    | table | postgres
 public | manejarmetro      | table | postgres
 public | manejarmetrobus   | table | postgres
 public | manejarmicrobuses | table | postgres
 public | manejarrtp        | table | postgres
 public | manejartaxi       | table | postgres
 public | manejartrenligero | table | postgres
 public | manejartrolebus   | table | postgres
 public | metodopago        | table | postgres
 public | metro             | table | postgres
```

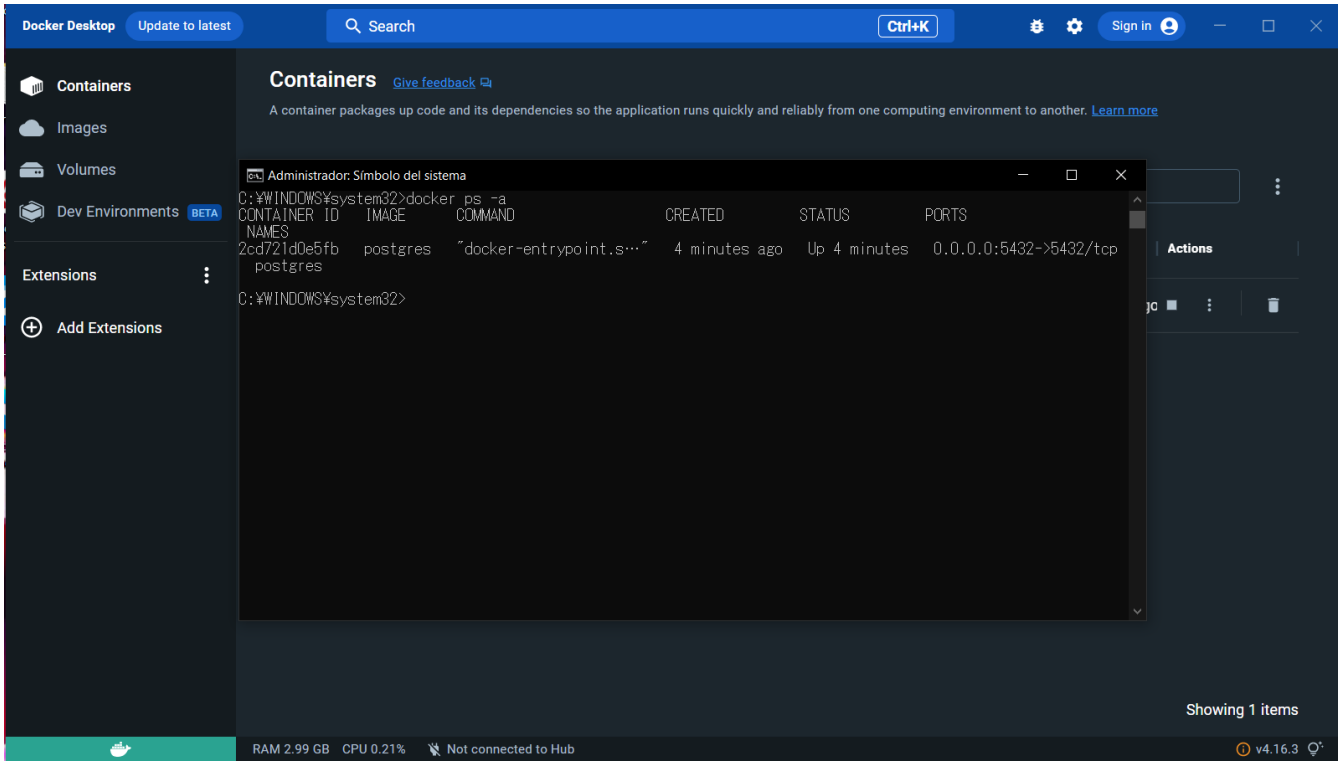
Y listo restauramos una base de datos a través de un backup.

Pasos para restaurar a partir de un sql: (Ejemplo Windows)

Son los mismos comandos utilizados en Linux.

i. Abrimos la terminal como administrador y checamos nuestros contenedores con:

```
docker ps -a
```



ii. Obtenemos el **CONTAINER ID** de la imagen de postgres, en este caso es *2cd721d0e5fb*

iii. Una vez que sepamos el **CONTAINER ID** de la imagen de postgres, debemos checar nuestras bases de datos primero, conectandonos a psql de postgres.

```
docker exec -it 2cd721d0e5fb psql -U postgres
```

iv. Una vez adentro de psql, escribimos lo siguiente:

```
\l
```

v. Con esto sabremos todas nuestras bases de datos, template0 y template1, son información que necesita postgres, diccionarios y eso. Una vez que checamos el nombre de nuestra base de datos salimos con:

```
\q
```

```
C:¥WINDOWS¥system32>docker exec -it 2cd721d0e5fb psql -U postgres
psql (15.2 (Debian 15.2-1.pgdg110+1))
Type "help" for help.

postgres=# ¥l
                                                List of databases
   Name    |  Owner   | Encoding |  Collate   |   Ctype    | ICU Locale | Locale Provider |   Access privi
leges
-----------+----------+----------+------------+------------+------------+-----------------+---------------
--------
 postgres  | postgres | UTF8     | en_US.utf8 | en_US.utf8 |            | libc            |
 template0 | postgres | UTF8     | en_US.utf8 | en_US.utf8 |            | libc            | =c/postgres
       +
           |          |          |            |            |            |                 | postgres=CTc/p
ostgres
 template1 | postgres | UTF8     | en_US.utf8 | en_US.utf8 |            | libc            | =c/postgres
       +
           |          |          |            |            |            |                 | postgres=CTc/p
ostgres
(3 rows)

postgres=# ¥q
```

vi. Para realizar el backup es necesario tener la base de datos vacia, por si se llega a tener tablas repetidas, para asegurarnos de eso crearemos una nueva base de datos el cual llamaremos ejemplo:

```
docker exec -it 2cd721d0e5fb createdb -U postgres ejemplo
```

vii. Una vez creado nos conectamos a esa nueva base:

```
docker exec -it 2cd721d0e5fb psql -U postgres ejemplo
```

viii. Checamos el contenido de la base con:

```
\dt
```

Deberia salirnos lo siguiente:

```
C:¥WINDOWS¥system32>docker exec -it 2cd721d0e5fb createdb -U postgres ejemplo

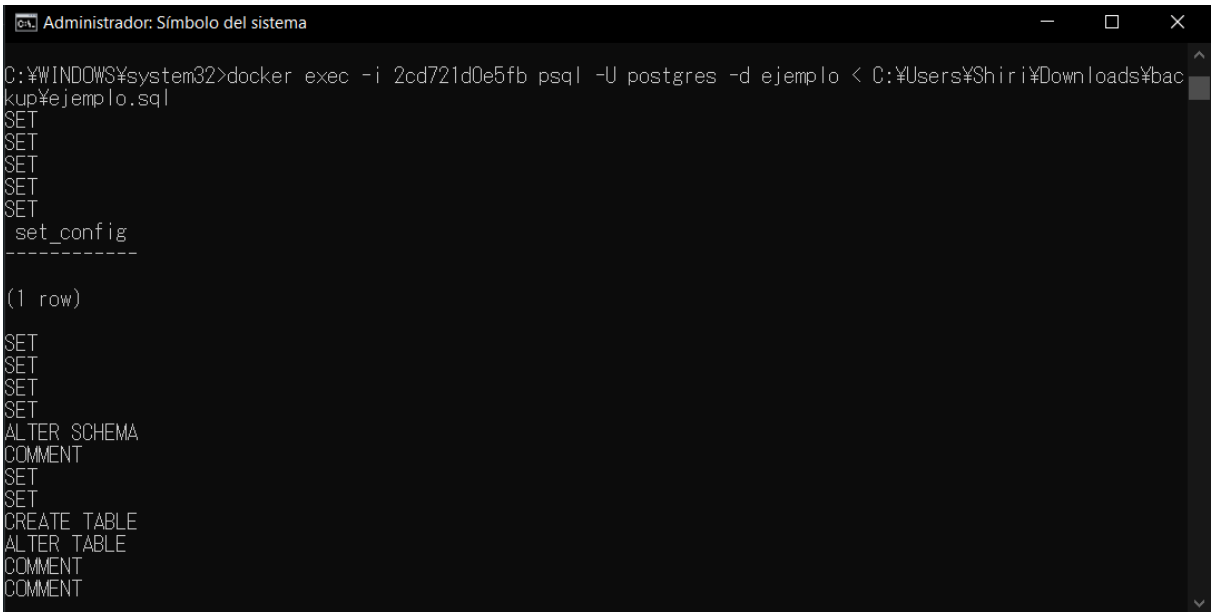
C:¥WINDOWS¥system32>docker exec -it 2cd721d0e5fb psql -U postgres ejemplo
psql (15.2 (Debian 15.2-1.pgdg110+1))
Type "help" for help.

ejemplo=# ¥dt
Did not find any relations.
ejemplo=#
¥q

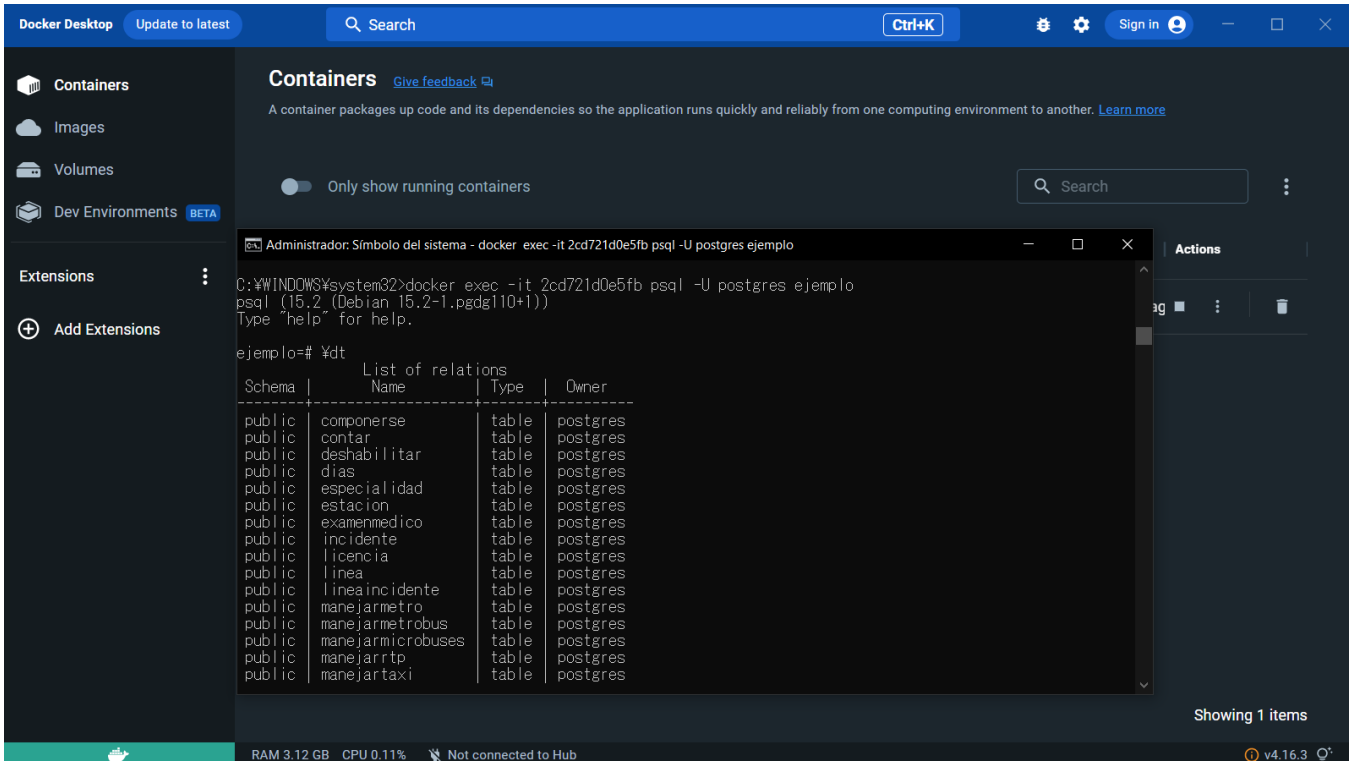
C:¥WINDOWS¥system32>
```

ix. Para realizar el backup utilizaremos < el cual tenemos que especificar la dirección donde se encuentra el backup.

```
docker exec -i 2cd721d0e5fb psql -U postgres -d ejemplo <
    C:\Users\Shiri\Downloads\backup\ejemplo.sql
```



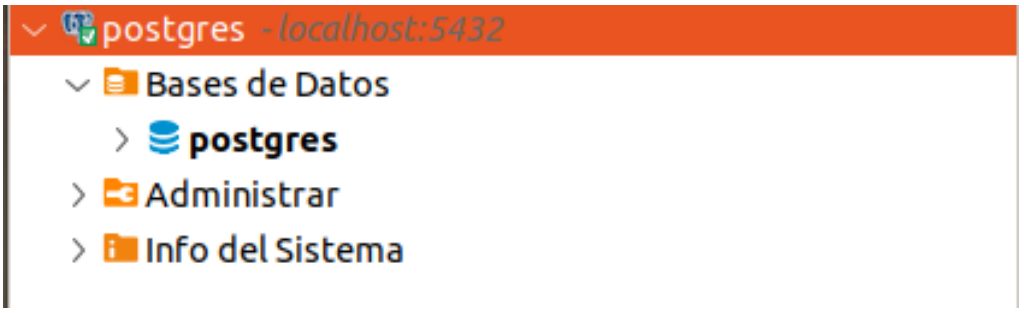
x. Si todo salio bien, al conectarnos a la base de datos ejemplo con psql y checar las tablas de nuestra base de datos, deberia aparecenos algo parecido:



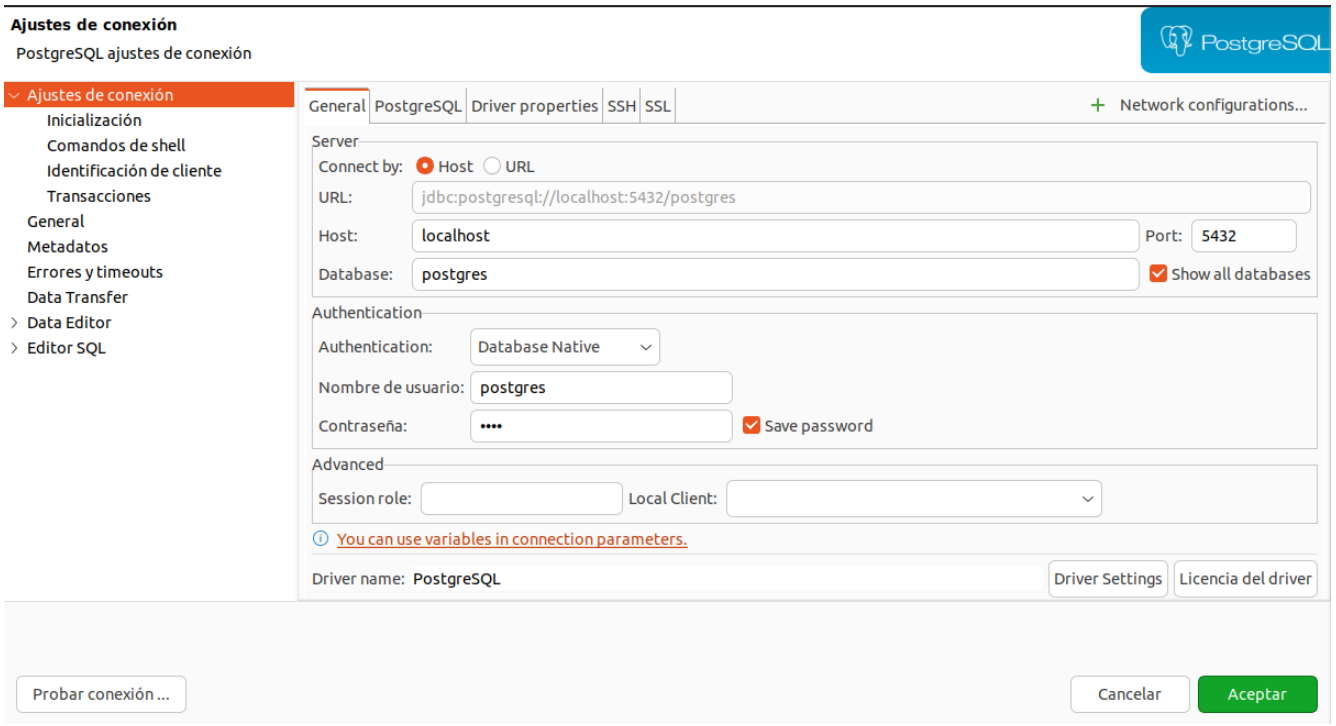
Y listo restauramos una base de datos a través de un backup en Windows.

Ver las bases que creamos en Dbeaver

Una vez que realizamos el backup es necesario configurar el **Database Tool DBeaver** para verlo. Ya que por predefinido solo veremos postgres.



i. Para activarlo damos click derecho sobre postgres - *localhost:5432* y seleccionamos **Editar Connection**. Y nos movemos a la pestanaña **General**
ii. Marcamos la opción **Show all database** y presionamos **Aceptar**.



iii. Nos dira "La conexión "postgres"ha sido modificada. ¿Quiere reconectarse?", seleccionamos si. Si todo salio bien deberia aparecernos todas nuestras bases de datos.



Restaurar un backup desde un dump file

Algunos backup, son realizados en formatos que no necesariamente son sql. Para estos casos debemos utilizar la instrucción **pg_restore**.

i. Son los mismos pasos que ya especificamos en el documento, la unica diferencia es que despues de la creación de la base de datos **ejemplo**. En vez de utilizar el comando para hacer el backup, utilizaremos pg_restore.

```
sudo docker exec -i 54eeed7dd8d8 pg_restore --verbose --clean --no-acl --no-owner -U
    postgres -d ejemplo < /home/ricardo/Descargas/mibanquito.backup
```

ii. Como se ve, es muy parecido al comando anterior, solo que esta vez se utilizo un .backup (pg_dump-file), en vez de un .sql.
Donde:
   - **–verbose, –clean, –no-acl, –no-owner**, son partes del comando pg_restore.
   - **verbose**: Permite imprimir en la termina el proceso del backup.
   - **clean**: Sobreescribe la información de la base de datos, con la información del backup.
   - **no-acl**: Previene la restauración de los privilegios de acceso.
   - **no-owner**: Evita problemas con los usuarios de los esquemas.

   Para mayor información: `https://www.postgresql.org/docs/current/app-pgrestore.html`.

iii. Si todo salio bien debe aparecer algo de este estilo.

```
n --no-acl --no-owner -U postgres -d ejemplo < /home/ricardo/Descargas/mibanquit
o.backup
pg_restore: connecting to database for restore
pg_restore: dropping FK CONSTRAINT fkprestatario2
pg_restore: dropping FK CONSTRAINT fkprestatario1
pg_restore: dropping FK CONSTRAINT fkprestamo
pg_restore: dropping FK CONSTRAINT fkpagos
pg_restore: dropping FK CONSTRAINT fkcuenta
pg_restore: dropping FK CONSTRAINT fkctacliente2
pg_restore: dropping FK CONSTRAINT fkctacliente1
pg_restore: dropping CONSTRAINT pksucursal
pg_restore: dropping CONSTRAINT pkprestamo
pg_restore: dropping CONSTRAINT pkcuenta
pg_restore: dropping CONSTRAINT pkcliente
pg_restore: dropping TABLE DATA sucursal
pg_restore: dropping TABLE DATA prestatario
pg_restore: dropping TABLE DATA prestamo
pg_restore: dropping TABLE DATA pagos
pg_restore: dropping TABLE DATA cuenta
pg_restore: dropping TABLE DATA ctacliente
pg_restore: dropping TABLE DATA cliente
pg_restore: dropping TABLE sucursal
pg_restore: dropping TABLE prestatario
pg_restore: dropping TABLE prestamo
```

iv. La primera vez que se ejecuta, puede que salga algunos errores, esto es debido a que el backup tiene instrucciones **DROP**. Al finalizar en **Dbeaver** se tiene que ver algo de este en la base de datos que creamos ejemplo.

